Perturbation Testing with Rate and Spiking RNNs: A Tutorial Reproduction with Learnable Time Constants

Perturbation Testing Tutorial

June 19, 2026

Abstract

This note documents a minimal tutorial pipeline for *perturbation testing* of recurrent network models of cortical spontaneous activity, following Sourmpis et al. (eLife 2026). We fit data-constrained recurrent neural networks (RNNs) to Neuropixels recordings from a single Allen Institute session and ask whether the fitted models predict the effect of optogenetic stimulation of parvalbumin-positive (PV) inhibitory neurons — a perturbation the model never saw during training. We summarise the four network architectures (a vanilla rate RNN, a leaky integrate-and-fire spiking RNN, and their low-rank inter-area variants), the biological constraints that can be switched on (Dale's law, E/I balance, spectral-radius control, inter-area Dale's law), the training objective (a trial-averaged plus a trial-matched loss on an oscillation-feature embedding), and the evaluation metrics (PSTH correlation, Brain-FID, and trial-matched R^2). We additionally describe a **membrane time constant** for the low-rank models, parameterised so it stays positive and bounded, which can be either learned jointly with the weights or held fixed, and is exposed as a swept hyperparameter in the experiment driver.

1 Overview and goal

The aim of perturbation testing is to evaluate a model not on the data distribution it was trained on, but on its response to a causal intervention. Here the intervention is optogenetic excitation of PV interneurons. A spontaneous-activity generator is trained *only* on unperturbed gray-screen activity; at test time the recorded optogenetic light waveform $i(t)$ is injected into the model's putative inhibitory units, and the model's predicted population response is compared against the held-out optogenetic recordings. A model that has captured the right recurrent structure should generalise to the perturbation; one that has merely memorised the baseline statistics should not.

This is a minimal reproduction of Sourmpis et al., eLife 2026 [1], using data from the Allen Institute Visual Coding Neuropixels dataset [3], session `829720705` (*Pvalb-Cre* × *Ai32*, functional-connectivity protocol), which combines PV optogenetic stimulation with simultaneous Neuropixels recordings. The trial-matching training loss follows Sourmpis et al., NeurIPS 2023 [2].

2 Data

Spike trains from QC-passing units across six visual areas (`VISp`, `VISrl`, `VISl`, `VISal`, `VISpm`, `VISam`) are binned at $\mathrm{dt} = 10\,\mathrm{ms}$. Neurons are sorted so they are grouped by area, and every neuron carries an area label and a sign label.

Conditions. Trials are integer-coded: $c = 0$ for spontaneous (gray-screen) windows, $c \geq 1$ for drifting gratings, and $c = -1$ for optogenetic-perturbation trials. Training and the in-distribution test set use a 70/30 stratified split per condition; the optogenetic trials form a separate perturbation set. The generators are trained on the spontaneous block only ($c = 0$), binarised to spikes.

Neuron signs. Each unit is labelled excitatory (+1), inhibitory (−1), or outlier (0) from a fast-spiking waveform classifier. Optionally the "ideal" PV labels can instead be derived directly from the perturbation itself (units whose firing rises during the high-light phase of the strongest raised-cosine trials; flag `--ideal-pv-neurons`). Inhibitory/PV units are the ones *driven* by the opto light at test time; they are excluded from scoring (the light clamps them identically in data and model, so scoring them would inflate the metrics).

Perturbation waveforms. The Allen functional-connectivity protocol provides three light waveforms — single `pulse`, `fast_pulses` (2.5 ms pulses at 10 Hz over 1 s), and `raised_cosine` — each at three intensity levels $\{1.3, 1.7, 2.0\}$. In-block `sham` segments (LED-free windows drawn from the gaps between light events) act as a matched control. Each perturbation trial is 1.5 s long with light onset at 0.5 s.

3 Models

All four models are *spontaneous-activity generators*: there is no external stimulus input; recurrent dynamics are driven only by per-neuron Gaussian noise. We write the recurrence as $\mathbf{u} = \mathbf{x}\mathbf{W}$, so W_{ij} is the weight from presynaptic i to postsynaptic j. All generators warm-start each call from the previous call's detached end state (no backpropagation across calls).

3.1 Vanilla rate RNN (`rnn`)

A ReLU-like rate unit clipped to $[0, 1]$:

$$\mathbf{u}_t = \mathbf{x}_{t-1}\mathbf{W} + \mathbf{b} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}_t + \mathbf{p}_t, \tag{1}$$

$$\mathbf{x}_t = \mathrm{clip}(\mathbf{u}_t,\, 0,\, 1), \tag{2}$$

where $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(0, I)$, $\boldsymbol{\sigma}$ is a learnable per-neuron noise scale (initialised to 0.1 so the noise-driven network does not collapse to the zero fixed point), and $\mathbf{p}_t$ is the optional perturbation current (zero during training). The output is scaled by a learnable scalar.

3.2 Leaky integrate-and-fire spiking RNN (`lif`)

A spiking variant with a leaky membrane and per-synapse transmission delays. Each synapse $i \to j$ is assigned at initialisation to one of `num_delays` delay bins via a one-hot tensor $\mathbf{W}^{(d)}$, so it reads its presynaptic spike from its own delay in the past:

$$\mathbf{u}_t = \textstyle\sum_k \big(\mathbf{z}_{t-k}\big)\big(W^{(d)}_{\cdot\cdot k} \odot \mathbf{W}\big) + \mathbf{b} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}_t + \mathbf{p}_t, \tag{3}$$

$$\mathbf{v}_t = \alpha\, \mathbf{v}_{t-1} + (1 - \alpha)\, \mathbf{u}_t - \mathbf{z}_{t-1}\, v_{\mathrm{thr}}, \tag{4}$$

$$p_t = \sigma_{\mathrm{logistic}}\big(\beta(\mathbf{v}_t - v_{\mathrm{thr}})\big), \qquad \mathbf{z}_t = \mathbb{1}[p_t > 0.5] + (p_t - \mathrm{sg}(p_t)), \tag{5}$$

where α is the membrane leak/retention factor, v_{thr} the firing threshold, and the last term is a straight-through estimator (forward: sampled 0/1 spike; backward: gradient through $p_t = \sigma_{\mathrm{logistic}}$). $\mathrm{sg}(\cdot)$ denotes stop-gradient.

3.3 Low-rank inter-area rate RNN (`lowRNN`)

This model imposes *area structure* on the connectivity. The global weight matrix is assembled block-wise: each intra-area block is a full $(N_a \times N_a)$ matrix, while each inter-area block is constrained to rank R:

$$\mathbf{W}^{(\text{tgt,src})} = \begin{cases} \mathbf{W}_{\text{full}} & \text{intra-area (tgt = src)}, \\ MN^\top & \text{inter-area, unconstrained}, \\ \mathbf{s}_{\text{src}} \odot \left(MN^\top\right) & \text{inter-area, Dale-constrained}, \end{cases} \qquad M \in \mathbb{R}^{N_{\text{src}} \times R},\ N \in \mathbb{R}^{N_{\text{tgt}} \times R}. \tag{6}$$

The dense matrix has its diagonal zeroed. Crucially, the `lowRNN` dynamics are now *leaky*, with a learnable time constant (Section 5):

$$\mathbf{x}_t = \alpha\, \mathbf{x}_{t-1} + (1-\alpha)\, \text{clip}(\mathbf{u}_t, 0, 1), \qquad \mathbf{u}_t = \mathbf{x}_{t-1}\mathbf{W}_{\text{global}} + \mathbf{b} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}_t + \mathbf{p}_t. \tag{7}$$

Setting $\alpha \to 0$ recovers the memoryless map $\mathbf{x}_t = \text{clip}(\mathbf{u}_t)$ of the vanilla rate RNN; $\alpha \to 1$ makes the rate change arbitrarily slowly.

3.4 Low-rank spiking RNN (`lowBio`)

The biological counterpart of `lowRNN`: it combines the low-rank/area-block connectivity of Section 3.3 with the delayed-synapse LIF dynamics of `lif`. Its membrane leak α is the same learnable, time-constant–derived quantity used by `lowRNN`, so its membrane time constant is learned and swept as well.

4 Biological constraints

The constraints below are applied at initialisation and re-projected after every optimiser step (`apply_constraint`).

Dale's law (sign constraint). With `--sign-constrained`, $\mathbf{W}$ is initialised sign-constrained and E/I-balanced per column (so the net presynaptic drive into each postsynaptic neuron is zero at init), and the signs are re-projected after each step (excitatory rows clamped ≥ 0, inhibitory rows ≤ 0). The construction follows the recipe of [2].

Self-connection removal. The diagonal of $\mathbf{W}$ is held at zero.

Spectral-radius control. Because the recurrence has little leak, an expansive $\mathbf{W}$ would blow the dynamics up over a trial. After each step the spectral radius is re-pinned to at most 1.2 (the low-rank factors M, N are scaled symmetrically by $\sqrt{\cdot}$ so $MN^\top$ scales correctly).

Inter-area Dale's law. For the low-rank models, `--inter-area-dale` additionally sign-constrains the inter-area blocks (the factors M, N are kept non-negative and the fixed row-sign vector $\mathbf{s}_{\text{src}}$ supplies the sign). Without it, intra-area blocks are sign-constrained but inter-area projections are free.

5 Membrane time constant: fixed or learnable

The membrane time constant τ of the low-rank models (`lowRNN` and `lowBio`) is parameterised in log-space so it is strictly positive for any real value, and the per-step retention/leak factor α used in Eqs. (7) is derived from it:

$$\tau = \exp(\theta), \qquad \alpha = \exp(-1/\tau) \in (0, 1), \tag{8}$$

where θ is the underlying scalar. Larger τ gives α closer to 1, i.e. longer memory and slower dynamics. The initial value τ_0 is set by the `--tau-init` flag (units of time bins). Because $\alpha \in (0, 1)$ and $\tau > 0$ are guaranteed by construction, no extra projection is needed in `apply_constraint`.

The time constant can be either **learnable** or **fixed**:

- *Learnable* (default): θ is a trained parameter optimised jointly with the weights; the network learns its own integration timescale, initialised at τ_0. The learned τ is logged each epoch.
- *Fixed* (`--fixed-tau`): θ is held constant as a non-trainable buffer, so $\tau \equiv \tau_0$ throughout training. Sweeping τ_0 in this mode performs a controlled scan over the time constant.

In the spiking `lowBio` model this same α is the membrane leak, so making it learnable directly learns (or, when fixed, prescribes) the neuronal integration timescale.

6 Training objective

Models are trained to match recorded spontaneous activity through two complementary losses computed on low-pass-filtered rasters (a length-5 moving average), then combined with coefficients λ_{ta} and λ_{tm}:

$$\mathcal{L} = \lambda_{\text{ta}}\, \mathcal{L}_{\text{ta}} + \lambda_{\text{tm}}\, \mathcal{L}_{\text{tm}}. \tag{9}$$

Trial-averaged loss $\mathcal{L}_{\text{ta}}$. The squared difference between the trial-averaged (PSTH) low-passed activity of generated and recorded batches. This roughly corresponds to the paper's per-neuron term.

Trial-matched loss $\mathcal{L}_{\text{tm}}$. Single trials carry variability that a PSTH loss ignores. Following [2], generated and recorded trials are embedded with a differentiable feature map $\phi(\cdot)$, an optimal one-to-one assignment between the two sets of trials is solved on the *detached* cost matrix (Hungarian algorithm), and the MSE over matched pairs is minimised (gradient flows through the matched feature entries).

Feature map ϕ. Both the differentiable loss (torch) and the numpy evaluation share a single feature pipeline so they cannot drift apart: (i) average activity within each area to obtain per-area population channels; (ii) project each channel onto a bank of log-spaced cosine/sine oscillation bands and take per-band amplitudes plus a DC term; (iii) z-score against the training-set statistics. Optimisation uses AdamW (learning rate 10^{-3}, weight decay 0.1); `apply_constraint` runs after each step. This is the "sample-and-measure" paradigm of [4]: generate freely, then penalise the dissimilarity between summary statistics of generated and recorded activity.

7 Theoretical background and related work

This section explains *why* the objective takes the form of Section 6, and in particular why the generators are trained free-running rather than autoregressively.

7.1 Generative distribution matching, not likelihood

The networks are *generative* models of spontaneous activity: noise is the only drive, and each `generate` call produces statistically independent trials. The training target is therefore the *distribution* of activity, not the next spike given the recorded past. The classical alternative — maximising the likelihood of the recorded spikes, as generalised linear models (GLMs) do — clamps the network's trajectory to the data and optimises one-step-ahead conditionals. As [4] showed, such likelihood-fit recurrent spiking networks "do not produce realistic neural activity" when run autonomously. The sample-and-measure approach instead back-propagates through stochastically simulated spike trains to match neuroscience summary statistics of the *freely generated* activity, which is exactly the regime used at test time.

7.2 Why two complementary losses

The two terms constrain different statistical moments and neither suffices alone:

- $\mathcal{L}_{\text{ta}}$ (the PSTH/$\mathcal{L}_{\text{neuron}}$ term) pins down *each neuron's* trial-averaged response — mean rate and time course. But because it averages over trials, a degenerate model that emits the mean PSTH on every trial (zero variability) scores perfectly; the loss is blind to trial-to-trial structure. [2] show PSTH-only models cannot reproduce, e.g., a bimodal single-trial distribution.
- $\mathcal{L}_{\text{tm}}$ (the trial-matching term) pins down the *distribution over single trials*. Since generated and recorded trials are unordered and independent ("no reason for generated trial k to correspond to recorded trial k"), it uses optimal transport: an assignment between the two trial sets in feature space (Hungarian algorithm on a *detached* cost, gradient through the matched entries) followed by a matched MSE. But on its own, the population-level features under-constrain individual neurons.

Together they yield correct single-neuron means *and* correct population trial-to-trial covariability. The eval-time Brain-FID (Section 9) is the non-differentiable, distribution-level analogue of $\mathcal{L}_{\text{tm}}$ on the same feature map.

7.3 Why free-running, not autoregressive / teacher-forced

Teacher forcing (feeding the recorded past as input) would be the autoregressive alternative. It is deliberately avoided here, for four reasons:

1. **Exposure bias.** A teacher-forced model conditions on *true* past states during training but on its *own* states at generation; the free-running dynamics it must ultimately produce are never directly optimised and can drift or destabilise. This train–test mismatch is the long-known motivation for fixes such as Professor Forcing [6] and scheduled sampling. Sample-and-measure sidesteps it by optimising the autonomous dynamics directly.
2. **The variability is the signal.** For spontaneous, noise-driven data there is no stimulus to "predict from". Teacher forcing would let the network lean on the clamped recorded past instead of learning the recurrent structure that *generates* the observed variability — which is precisely the scientific object of interest.

3. **Perturbation testing requires autonomy.** The end goal is a counterfactual: inject the optogenetic current into PV units and predict the response. In that counterfactual there are *no* recorded spikes to clamp to, so the model must run autonomously. A teacher-forced model has no validated free-running behaviour and cannot be used for this prediction.

4. **Hidden neurons.** Clamping requires observing every neuron; sample-and-measure handles unobserved/hidden units naturally [4].

Note the code still uses back-propagation *within* a trial; it merely warm-starts each call from the *detached* previous end state (`self._state = x.detach()`) and never conditions on recorded spikes, so there is no gradient coupling across calls and no teacher forcing.

7.4 Related work

GLM and maximum-likelihood recurrent spiking networks fit one-step conditionals with clamped trajectories and are the teacher-forced baseline criticised above. [4] introduced the free-running sample-and-measure paradigm; [2] added the optimal-transport trial-matching loss to capture variability; the present pipeline follows [1], which applies these models to optogenetic perturbation prediction. A different lineage — sequential variational auto-encoders such as LFADS [5] — uses amortised inference to recover per-trial initial conditions and inferred inputs and *reconstructs* observed trials; it excels at single-trial denoising but is not a pure free-running generator matched at the distribution level, and its inferred inputs complicate clean perturbation counterfactuals. Distribution-level generative models of spikes (GANs, latent diffusion) share the "match the distribution" goal with very different, less mechanistic machinery.

8 Autoregressive training mode (alternative objective)

As a deliberate counterpoint to the free-running, distribution-matching objective of Section 6, the pipeline also offers an **autoregressive roll-out** objective, selected with `--train-mode autoregressive` (the default remains `trialmatch`). Here the network state is initialised *from data* and rolled out open-loop, and the prediction is matched to the *same* trials — so trial correspondence is given by construction and no optimal-transport assignment is needed.

8.1 Procedure

For a recorded trial $\mathbf{z}^{\mathcal{D}}$ and a start bin t_0, the state is seeded from the data and the model free-runs for R steps (`--rollout-len`), producing $\hat{\mathbf{z}}_{t_0:t_0+R}$:

- *Rate models* (`rnn`, `lowRNN`): the rate state is set to the last observed bin, $\mathbf{x}_{t_0-1} = \mathbf{z}^{\mathcal{D}}_{t_0-1}$.
- *Spiking models* (`lif`, `lowBio`): the delayed-spike buffer is filled with the last `num_delays` recorded bins and the membrane is started at $\mathbf{v} = 0$ (an approximation; the first steps are a transient).

Several start bins t_0 are sampled *within* each trial per batch (`--n-inits`), yielding many initial conditions. The roll-out is open-loop (the network feeds back its own output — no teacher forcing during the roll-out); only the initial condition comes from data. As in the free-running mode, gradients flow within the roll-out but the persistent warm-start buffer is left untouched.

8.2 Losses

The trial-matching term $\mathcal{L}_{\text{tm}}$ is dropped. The objective is

$$\mathcal{L} = \lambda_{\text{ta}}\,\mathcal{L}_{\text{ta}} + \lambda_{\text{mse}}\,\mathcal{L}_{\text{roll}} + \lambda_{\text{band}}\,\mathcal{L}_{\text{band}}, \tag{10}$$

where (i) $\mathcal{L}_{\text{ta}}$ is the same free-running trial-averaged PSTH loss as before (it still constrains global per-neuron means); (ii) $\mathcal{L}_{\text{roll}}$ is the element-wise MSE between the low-pass-filtered roll-out and the matched data segment, $\| \operatorname{lp}(\hat{\mathbf{z}}) - \operatorname{lp}(\mathbf{z}^{\mathcal{D}}_{t_0:t_0+R}) \|^2$; and (iii) $\mathcal{L}_{\text{band}}$ is the MSE between the oscillation-band features ϕ of the roll-out and of the *same* data segment — the same embedding as $\mathcal{L}_{\text{tm}}$ but with the trials matched by construction (no Hungarian step). Coefficients are `--coeff_ta`, `--coeff_ar_mse`, `--coeff_ar_band`. Because the roll-out is stochastic, the smoothing keeps $\mathcal{L}_{\text{roll}}$ meaningful only for the low-frequency, short-horizon component; the band term carries the oscillation content over the window. Since the features are integrated over the R-bin window, their normalisation statistics are refit on R-length crops of the training data so the lowest bands remain comparable.

8.3 Effect on metrics and plots

The free-running generative metrics of Section 9 (PSTH-r, Brain-FID, optimal-transport R^2) are still computed each epoch. In addition, a **matched roll-out** evaluation is reported: test trials are seeded from data, rolled out for R steps, and scored against the matched data segment with the trial order preserved (the `matched`=True path), giving a matched roll-out R^2 that becomes the primary tracked quantity and adds a fourth panel to the over-epochs figure. The training raster switches from free-running samples to a *matched prediction*: each data trial is shown beside its own roll-out (data prefix followed by the open-loop continuation). The perturbation-testing evaluation is **unchanged** — it remains free-running with the injected opto current, since the counterfactual has no spikes to seed from (Section 7).

9 Evaluation metrics

Three metrics compare model-generated rasters to held-out recorded rasters of the same shape:

1. **PSTH Pearson** r (↑): per-neuron Pearson correlation between trial-averaged firing rates (model vs. data). Trials need not be matched.
2. **Brain-FID** (↓): Fréchet distance between Gaussian fits to the distribution of single-trial feature vectors $\phi(z)$ of model vs. data (a distribution-level score, in the spirit of the image FID).
3. **Trial-matched** R^2 (↑): R^2 between optimally matched single-trial populations on the same feature embedding ϕ used by Brain-FID.

Two reference levels frame the scores: *chance baselines* from time-shuffled and neuron-shuffled rasters, and an *ideal ceiling* from comparing real train data against real test data. Brain-FID and trial-matched R^2 deliberately share the feature map ϕ so the two numbers are consistent.

Perturbation evaluation. After training, the recorded light waveform $i(t)$ for each condition is injected as a perturbation current into the driven (PV/inhibitory) units, scaled by an *opto intensity* factor that is swept over `--opto-intensities`. Metrics are computed on the non-driven units only. Conditions span `sham` and every (waveform, level) combination present in the session.

Flag	Meaning	Default
`--model`	`rnn` / `lif` / `lowRNN` / `lowBio`	`rnn`
`--sign-constrained`	enable Dale's law (sign-constrained, E/I-balanced $\mathbf{W}$)	off
`--inter-area-dale`	sign-constrain inter-area blocks (low-rank models)	off
`--rank`	inter-area low-rank R (low-rank models)	2
`--tau-init`	**initial/fixed membrane time constant (bins)**	2.0
`--fixed-tau`	**hold τ fixed at `--tau-init` instead of learning it**	off (learnable)
`--num-delays`	number of synaptic delay bins (spiking models)	3
`--ideal-pv-neurons`	use perturbation-derived PV labels instead of waveform	off
`--epochs`	training epochs	20
`--batch-size`	trials per batch	64
`--lr`	AdamW learning rate	10^{-3}
`--coeff_ta`	weight λ_{ta} of the trial-averaged loss	1.0
`--coeff_tm`	weight λ_{tm} of the trial-matched loss	0.3
`--train-mode`	**`trialmatch` / `autoregressive`** (Section 8)	`trialmatch`
`--rollout-len`	**roll-out length R in bins (autoregressive)**	50
`--n-inits`	**within-trial initial conditions per batch (autoregressive)**	4
`--coeff_ar_mse`	**weight λ_{mse} of the roll-out MSE (autoregressive)**	1.0
`--coeff_ar_band`	**weight λ_{band} of the matched-band MSE (autoregressive)**	0.3
`--opto-intensities`	perturbation-current scale factors (swept at eval)	0.1 0.5 1.0

Table 1: Command-line settings of the training script. The time constant (`--tau-init`) is the newly added learnable, swept parameter.

Model family	Swept axes	Runs
`rnn`	{unconstrained, sign-constrained}	2
`lif`	{unconstrained, sign-constrained}	2
`lowRNN`, `lowBio`	{unconstr., sign-constr., inter-Dale} $\times$ rank$\{1,2,3,4\}$ $\times$ $\tau_0\{1,2,4,8\}$	$2 \times 3 \times 4 \times 4 = 96$
Total		100

Table 2: Sweep performed by `run_baselines.sh`. The low-rank families now sweep the time-constant initialisation τ_0 in addition to rank and the constraint configuration. Each run additionally evaluates every opto-intensity in `--opto-intensities`. Results are written to `figures/<model>_<constraint>[_rank<R>][_tau<T>]/`.

10 Settings and experiment sweep

Table 1 lists the command-line settings of `train_rnn.py`. Table 2 summarises the full sweep run by `run_baselines.sh`.

11 Conclusion

The tutorial provides a compact, end-to-end perturbation-testing pipeline: data-constrained rate and spiking RNNs, optional biological constraints, a trial-matching training objective, and distribution-level metrics evaluated under a held-out optogenetic perturbation. The low-rank, area-structured models (`lowRNN`, `lowBio`) now carry a learnable membrane time constant that is both fit by gradient descent and swept across initialisations, letting the neuronal integration timescale be studied as a first-class hyperparameter alongside connectivity rank and Dale's-law constraints.

References

[1] G. Sourmpis et al. Perturbation testing of data-constrained network models. *eLife*, 2026. https://elifesciences.org/articles/106827.

[2] G. Sourmpis, C. Petersen, W. Gerstner, and G. Bellec. Trial matching: capturing variability with data-constrained spiking neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://papers.neurips.cc/paper_files/paper/2023/hash/ec702dd6e83b2113a43614685a7e2ac6-Abstract-Conference.html.

[3] J. H. Siegle et al. Survey of spiking in the mouse visual system reveals functional hierarchy. *Nature*, 592:86–92, 2021. https://www.nature.com/articles/s41586-020-03171-x.

[4] G. Bellec, S. Wang, A. Modirshanechi, J. Brea, and W. Gerstner. Fitting summary statistics of neural data with a differentiable spiking network simulator. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. https://arxiv.org/abs/2106.10064.

[5] C. Pandarinath et al. Inferring single-trial neural population dynamics using sequential auto-encoders (LFADS). *Nature Methods*, 15:805–815, 2018. https://www.nature.com/articles/s41592-018-0109-9.

[6] A. Lamb, A. Goyal, Y. Zhang, S. Zhang, A. Courville, and Y. Bengio. Professor Forcing: a new algorithm for training recurrent networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. https://arxiv.org/abs/1610.09038.